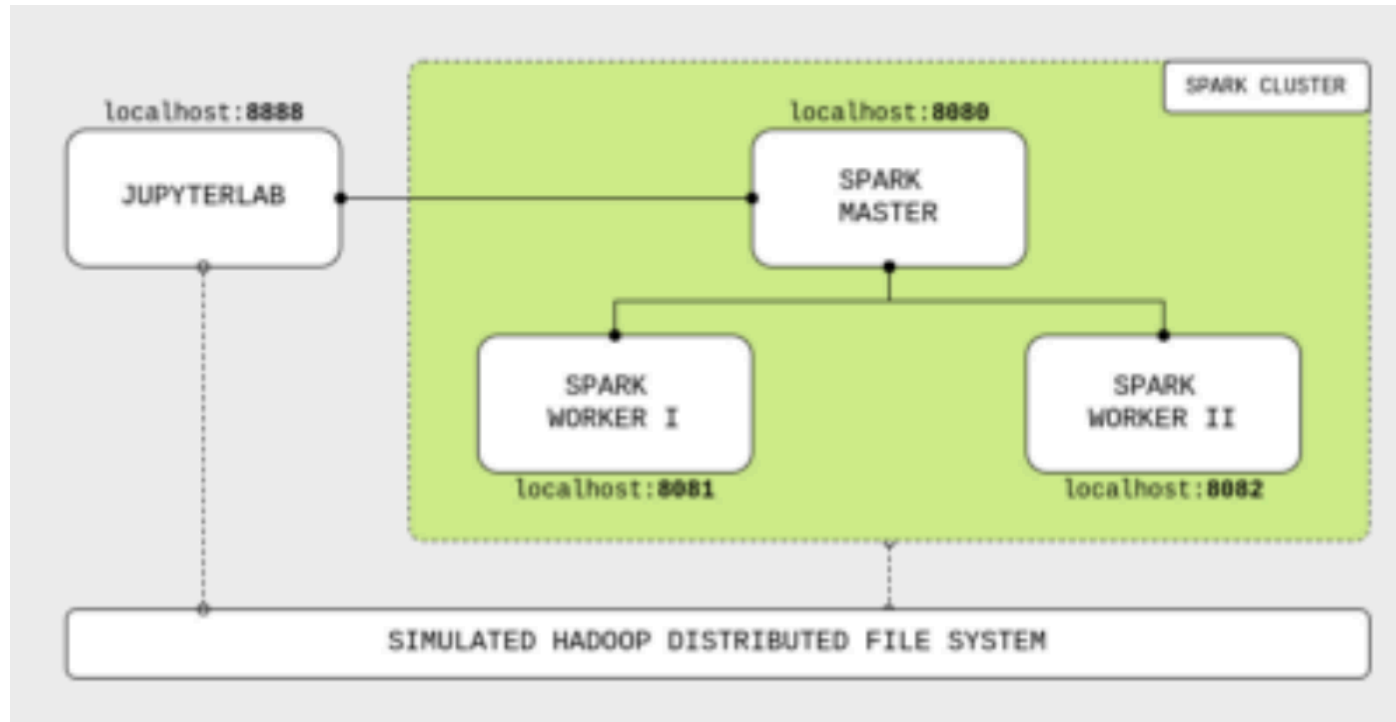
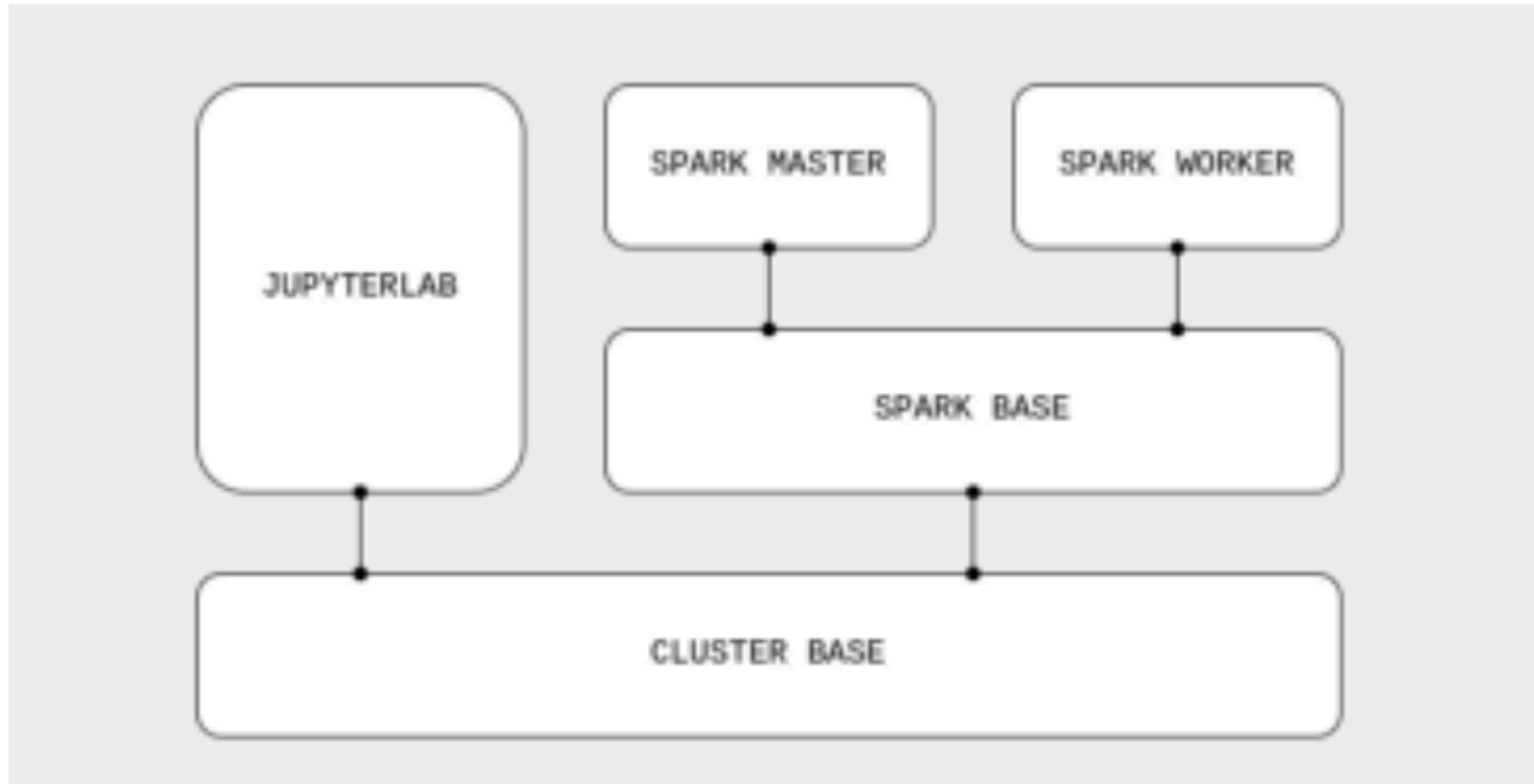


Spark 구성도

- 먼저 Spark Master를 생성해 주고 이를 Localhost에 8080 Port에 엮어 준다.
- 그 다음 두개의 Worker를 구성하여 각각 8081 / 8082 Port에 엮어 준다.
- 그리고 Spark Code를 원활하게 실행할 JupyterLab을 8888포트에 올려 준비한다.



Docker 구성도



Docker File 생성

단계1: cluster-base

```
ARG debian_buster_image_tag=8-jre-slim
FROM openjdk:${debian_buster_image_tag}

# -- Layer: OS + Python 3.7

ARG shared_workspace=/opt/workspace

RUN mkdir -p ${shared_workspace} && \
    apt-get update -y && \
    apt-get install -y python3 && \
    apt-get install -y python3-pip && \
    ln -s /usr/bin/python3 /usr/bin/python && \
    rm -rf /var/lib/apt/lists/*

ENV SHARED_WORKSPACE=${shared_workspace}

# -- Runtime

VOLUME ${shared_workspace}
CMD ["bash"]
```

단계2: spark-base

```
FROM cluster-base

# -- Layer: Apache Spark

ARG spark_version=3.1.1
ARG hadoop_version=3.2

RUN apt-get update -y && \
    pip3 install --upgrade pip setuptools wheel && \
    pip3 install pandas && \
    apt-get install -y curl && \
    curl https://archive.apache.org/dist/spark/spark-${spark_version}/spark-${spark_version}-bin-hadoop${hadoop_version}.tgz -o spark.tgz && \
    tar -xf spark.tgz && \
    mv spark-${spark_version}-bin-hadoop${hadoop_version} /usr/bin/ && \
    mkdir /usr/bin/spark-${spark_version}-bin-hadoop${hadoop_version}/logs && \
    rm spark.tgz

ENV SPARK_HOME /usr/bin/spark-${spark_version}-bin-hadoop${hadoop_version}
ENV SPARK_MASTER_HOST spark-master
ENV SPARK_MASTER_PORT 7077
ENV PYSPARK_PYTHON python3

# -- Runtime

WORKDIR ${SPARK_HOME}
```

단계3: spark-master

```
FROM spark-base

# -- Runtime

ARG spark_master_web_ui=8080

EXPOSE ${spark_master_web_ui} ${SPARK_MASTER_PORT}
CMD bin/spark-class org.apache.spark.deploy.master.Master >> logs/spark-master.out
```

단계4: spark-worker

```
FROM spark-base

# -- Runtime

ARG spark_worker_web_ui=8081

EXPOSE ${spark_worker_web_ui}
CMD bin/spark-class org.apache.spark.deploy.worker.Worker spark://${SPARK_MASTER_HOST}:${SPARK_MASTER_PORT} >> logs/spark-worker.out
```

단계5: jupyterlab

```
FROM cluster-base

# -- Layer: JupyterLab

ARG spark_version=3.1.1
ARG jupyterlab_version=2.1.5

RUN apt-get update -y && \
    apt-get install -y python3-pip && \
    pip3 install --upgrade pip setuptools wheel && \
    pip3 install pandas && \
    pip3 install wget pyspark==${spark_version} jupyterlab==${jupyterlab_version}

# -- Runtime

EXPOSE 8888
WORKDIR ${SHARED_WORKSPACE}
CMD jupyter lab --ip=0.0.0.0 --port=8888 --no-browser --allow-root --NotebookApp.token=
```

Docker Image 생성

```
# -- Building the Images
```

```
docker build -f cluster-base.Dockerfile -t cluster-base .
```

```
docker build --build-arg spark_version="3.1.1" --build-arg hadoop_version="3.2" -f spark-base.Dockerfile -t spark-base .
```

```
docker build -f spark-master.Dockerfile -t spark-master .
```

```
docker build -f spark-worker.Dockerfile -t spark-worker .
```

```
docker build --build-arg spark_version="3.1.1" --build-arg jupyterlab_version="2.1.5" -f jupyterlab.Dockerfile -t jupyterlab .
```

Docker-compose

단계1: Docker-compose file 생성

```
version: "3.6"
volumes:
  shared-workspace:
    name: "hadoop-distributed-file-system"
    driver: local
services:
  jupyterlab:
    image: jupyterlab
    container_name: jupyterlab
    ports:
      - 8888:8888
    volumes:
      - shared-workspace:/opt/workspace
...
```

```
...
spark-master:
  image: spark-master
  container_name: spark-master
  ports:
    - 8080:8080
    - 7077:7077
  volumes:
    - shared-workspace:/opt/workspace
spark-worker-1:
  image: spark-worker
  container_name: spark-worker-1
  environment:
    SPARK_WORKER_CORES: "1"
    SPARK_WORKER_MEMORY: "1024m"
  ports:
    - 8081:8081
  volumes:
    - shared-workspace:/opt/workspace
  depends_on:
    - spark-master
spark-worker-2:
  image: spark-worker
  container_name: spark-worker-2
  environment:
    SPARK_WORKER_CORES: "1"
    SPARK_WORKER_MEMORY: "1024m"
  ports:
    - 8082:8081
  volumes:
    - shared-workspace:/opt/workspace
  depends_on:
    - spark-master
```

단계2: Docker-compose 실행

```
docker-compose up -d
```

```
# 결과 확인
```

```
docker images
```

```
PS C:\dev\docker\spark_cluster> docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
apache-image	latest	4a272f419fa1	28 minutes ago	497MB
jupyterlab	latest	5702dd6e355e	About an hour ago	1.6GB
spark-master	latest	e7cfc6eae282	About an hour ago	1.02GB
spark-worker	latest	2681cac4b9f5	About an hour ago	1.02GB
spark-base	latest	b9b2d77f5f0d	About an hour ago	1.02GB
cluster-base	latest	86b3a8d93b64	2 hours ago	550MB
nginx	latest	c613f16b6642	2 months ago	187MB

```
PS C:\dev\docker\spark_cluster>
```

단계3: docker ps

```
docker ps
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE CODE REFERENCE LOG powershell - spark_clus
```

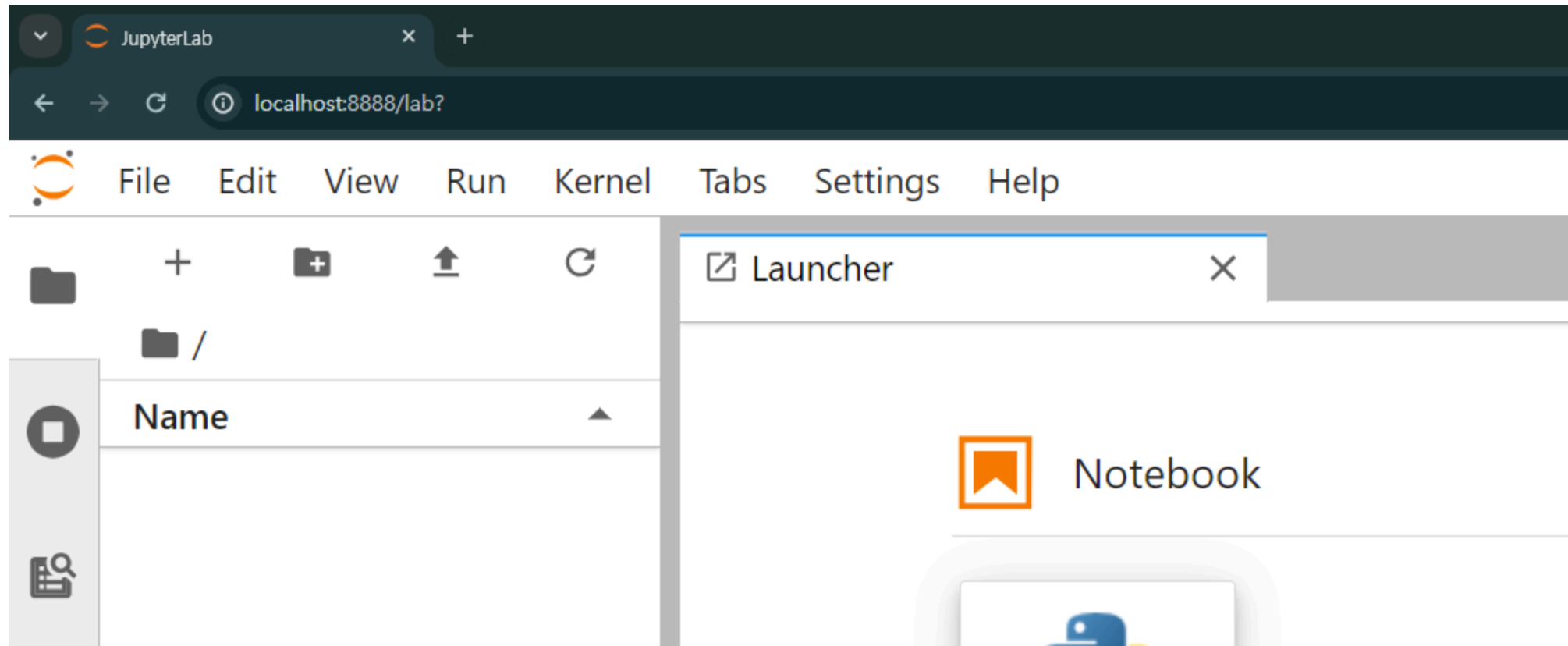
```
PS C:\dev\docker\spark_cluster> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
d8e6f7e19e98	spark-worker	"/bin/sh -c 'bin/spa..."	11 seconds ago	Up 9 seconds	0.0.0.0:8081->8081/tcp	spark-worker-1
96d0ee9533eb	spark-worker	"/bin/sh -c 'bin/spa..."	11 seconds ago	Up 9 seconds	0.0.0.0:8082->8081/tcp	spark-worker-2
e19179cc6b0b	jupyterlab	"/bin/sh -c 'jupyter..."	11 seconds ago	Up 10 seconds	0.0.0.0:8888->8888/tcp	jupyterlab
0cba0f56f2fe	spark-master	"/bin/sh -c 'bin/spa..."	11 seconds ago	Up 10 seconds	0.0.0.0:7077->7077/tcp, 0.0.0.0:8080->8080/tcp	spark-master

```
PS C:\dev\docker\spark_cluster>
```

단계4: jupyter Lab 접속

- localhost:8888



PySpark Test

단계1: SparkSession

```
from pyspark.sql import SparkSession

spark = SparkSession.\
    builder.\
    appName("pyspark-notebook").\
    master("spark://spark-master:7077").\
    config("spark.executor.memory", "512m").\
    getOrCreate()
```

Untitled.ipynb



Code



Python 3 (ipykernel) (

```
[1]: from pyspark.sql import SparkSession
```

```
[8]: spark = SparkSession.\
      builder.\
      appName("pyspark-notebook").\
      master("spark://spark-master:7077").\
      config("spark.executor.memory", "512m").\
      getOrCreate()
```

```
24/04/18 05:18:28 WARN NativeCodeLoader: Unable to load native-hadoop library for your platfo
rm... using builtin-java classes where applicable
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
```

단계2: Download Data

```
import wget
```

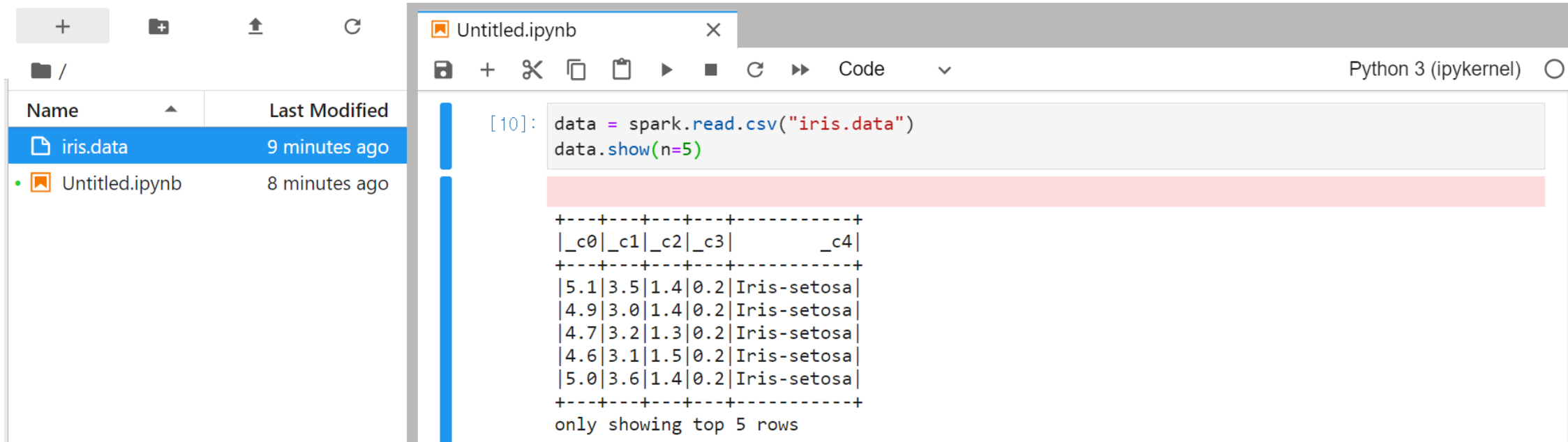
```
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"  
wget.download(url)
```

The screenshot displays a Jupyter Notebook environment. On the left, a file browser pane shows a directory with two files: 'iris.data' (modified 8 minutes ago) and 'Untitled.ipynb' (modified 7 minutes ago). The 'iris.data' file is highlighted. The main notebook area shows a tab for 'Untitled.ipynb' with a toolbar and a 'Code' dropdown menu. The notebook content includes a warning message about the native-hadoop library, followed by a code cell with the same Python code as shown in the previous blocks. The code cell is numbered [9]: and the output below it shows 'iris (1).data'.

```
24/04/18 05:18:28 WARN NativeCodeLoader: Unable to load native-hadoop library for your platfo  
rm... using builtin-java classes where applicable  
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties  
Setting default log level to "WARN".  
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).  
  
[9]: import wget  
  
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"  
wget.download(url)  
  
[9]: 'iris (1).data'
```

단계3: Show Data

```
data = spark.read.csv("iris.data")
data.show(n=5)
```



The screenshot shows a Jupyter Notebook interface. On the left is a file explorer with a table of files:

Name	Last Modified
iris.data	9 minutes ago
Untitled.ipynb	8 minutes ago

The main area shows a code cell for 'Untitled.ipynb' with the following code:

```
[10]: data = spark.read.csv("iris.data")
      data.show(n=5)
```

The output of the code cell is a table with 5 rows of data:

_c0	_c1	_c2	_c3	_c4
5.1	3.5	1.4	0.2	Iris-setosa
4.9	3.0	1.4	0.2	Iris-setosa
4.7	3.2	1.3	0.2	Iris-setosa
4.6	3.1	1.5	0.2	Iris-setosa
5.0	3.6	1.4	0.2	Iris-setosa

only showing top 5 rows

참고문서

- <https://todaycodeplus.tistory.com/31>